

The Aegis Knowledge Ingestion Challenge

1. Dataset Specification — "Aegis Series-7 Hydraulic Control System"

You will receive a fictional, internally-consistent-but-messy documentation package for a fictional industrial system (the "Aegis Series-7 HCS"). Fictional so it can't be solved by searching GitHub or a real product's public docs. 20 files, mixed formats, mixed reliability, mixed modality.

- aegis-dataset/
 - manuals/
 - operator_manual.pdf (12 pp, born-digital, clean layout)
 - maintenance_manual.pdf (18 pp, born-digital, includes tables + warnings)
 - legacy_manual_v1.html (exported from an old CMS, v1 of the operator manual – PARTIALLY SUPERSEDED)
 - engineering_bulletins/
 - ECN-1042.pdf (2 pp – sensor replacement + pressure spec change)
 - ECN-1058.pdf (1 p – alarm threshold correction, references ECN-1042)
 - reference/
 - alarm_reference.pdf (6 pp – multi-row header table, merged cells, footnotes)
 - component_register.xlsx (component IDs, aliases, locations – inconsistent ID formatting)
 - terminology_glossary.docx (official term list – INCOMPLETE, doesn't cover all aliases actually used elsewhere)
 - revision_history.xlsx (software/hardware revision timeline, effective dates)
 - diagrams/
 - system_diagram_hydraulic.pdf (schematic, topology only recoverable from shapes/lines, not text)
 - system_diagram_electrical.png (raster, labels partially low-contrast/rotated)
 - wiring_diagram.pdf (cross-references component IDs used nowhere else verbatim)
 - screenshots/
 - screen_01_home.png (HMI home screen, shows live-style values baked into image)
 - screen_02_alarms.png (HMI alarm list screen)
 - screen_03_diagnostics.png (HMI diagnostics screen, shows a sensor ID variant)
 - scans/

- scanned_appendix_calibration.pdf (scanned, skewed, moderate OCR noise, calibration table)
- configuration/
 - configuration_export.json (machine config dump – real structured data, but keys don't match doc terminology 1:1)
- low_trust/
 - site_survey_notes.docx (informal field engineer notes – free text, contradicts one manual value, no revision control)
- noise/
 - safety_data_sheet_hydraulic_fluid.pdf (real-looking MSDS, LARGELY IRRELEVANT to the Q&A set – tests noise filtering)
- extra/
 - training_slide_excerpt.pptx (3 slides, restates operator manual content in a different structure/wording)

A total of 20 files across 8 formats (PDF born-digital, PDF scanned, HTML, XLSX, DOCX, PNG raster diagrams/screenshots, JSON, PPTX) and at least 4 distinct trust levels (official manual, engineering bulletin/change notice, field notes, irrelevant reference doc).

2. Task Statement

Aegis Knowledge Ingestion Challenge

You are given `aegis-dataset/`: a heterogeneous, messy, partially contradictory documentation package for a fictional industrial control system (the Aegis Series-7 HCS). Some of it is well-structured, some is scanned and noisy, some is informal and untrustworthy, and some of it is irrelevant.

Your task: Build a system that transforms this raw material into a structured, provenance-preserving knowledge representation, and expose an interface through which an AI agent can answer natural-language questions **with evidence**, while correctly expressing uncertainty, conflicts, and gaps rather than guessing.

The agent on top should be the easiest part of this system. The hard part — and the part we are evaluating — is everything underneath it: how you read the sources, what representation you design, how you extract facts, how you resolve entities and identifiers, how you preserve where every fact came from, and how you handle the cases where sources disagree or say nothing at all.

You must:

1. Ingest all file types in the package (PDF, scanned PDF, HTML, XLSX, DOCX, PNG, JSON, PPTX).

2. Design and document your own intermediate knowledge representation. You are free to design the schema yourselves.
3. Build an extraction pipeline that is explicit about which steps are deterministic/rule-based vs. model-based, etc., and why.
4. Resolve entity aliases and identifier variants (e.g., decide whether two labels refer to the same real-world thing) and justify non-obvious decisions.
5. Represent conflicting or version-scoped facts explicitly (not by silently picking one value).
6. Attach provenance (source document, page/section/location) to every extracted fact used in an answer.
7. Expose a query interface or a User Interface that returns an answer, the claims it relies on, their evidence, and anything it could not determine.
8. Build a small evaluation harness against the question set provided for you in **evaluation_questions.pdf** and report quantitative results (not just example transcripts).

You may use AI coding assistants (Cursor, Claude Code, Copilot, etc.) freely. We care about your reasoning and design decisions, not whether you typed every line yourself. Be ready to explain and defend every non-trivial choice, and maybe to modify the system live in a follow-up session.

Deliverables:

- Source code + a **README.md** explaining your architecture, your representation, what you chose not to do and why, and your loss/confidence analysis.
- Results of the question set (metrics, not just transcripts). Find the questions in **evaluation_questions.pdf**
- Architecture design report.